Artificial Intelligence Lab (CAI601400)

Lab 5: Adversarial Search — Minimax & Alpha-Beta Pruning

Student Name: Ali Hassan	**Roll No:** SU92-BSAIM-F25-338
Program: BS Artificial Intelligence	**Week:** 5 (Lectures L11–L12)

Objective

To apply the Minimax algorithm and Alpha-Beta pruning on a game tree, compute the optimal value for the maximizing player, and show that pruning reduces the number of leaves evaluated without changing the Minimax decision.

Task 1 — Hand Trace of the Game Tree (Minimax)

In a game tree, the root is a MAX node (the player who wants the highest score), the next level is MIN (the opponent, who wants the lowest score), and the levels keep alternating. Minimax works bottom-up: each MAX node takes the maximum of its children and each MIN node takes the minimum.

Tree A — scores [3, 5, 2, 9, 0, 7], padded with zeros to a complete tree: [3, 5, 2, 9, 0, 7, 0, 0]

Level	How the values are obtained	Values
Leaves	Given terminal scores	3 5 2 9 0 7 0 0
MAX (depth 2)	max of each pair	max(3,5)=5 max(2,9)=9 max(0,7)=7 max(0,0)=0
MIN (depth 1)	min of each pair	min(5,9)=5 min(7,0)=0
Root — MAX	max of the two MIN values	max(5, 0) = **5**

So the optimal value for MAX is **5**, and the best first move is the left branch (the one whose MIN value is 5).

Tree B — scores [3, 5, 2, 9, 3, 5, 2, 9] (the tree used for alpha-beta)

Level	How the values are obtained	Values
Leaves	Given terminal scores	3 5 2 9 3 5 2 9
MAX (depth 2)	max of each pair	5 9 5 9
MIN (depth 1)	min of each pair	5 5
Root — MAX	max of the two MIN values	max(5, 5) = **5**

Again the optimal value is **5**. Both trees agree, which is what the reference check requires.

Task 2 — Leaf-Evaluation Counter

To compare the work done by the two algorithms, a counter is added at the exact point where a leaf (terminal node) is reached. Every time the recursion reaches the target depth, one leaf has been evaluated, so the counter goes up by one. Below is Minimax with the counter added (the two counter lines are marked):

```
import math

leaf_count = 0            # counts evaluated leaves

def minimax(curDepth, nodeIndex, maxTurn, scores, targetDepth):
```

```
    global leaf_count
    if curDepth == targetDepth:
        leaf_count += 1   # <-- counter: one leaf evaluated
        return scores[nodeIndex]
    if maxTurn:
        return max(minimax(curDepth + 1, nodeIndex * 2, False, scores, targetDepth),
                   minimax(curDepth + 1, nodeIndex * 2 + 1, False, scores, targetDepth
))
    else:
        return min(minimax(curDepth + 1, nodeIndex * 2, True, scores, targetDepth),
                   minimax(curDepth + 1, nodeIndex * 2 + 1, True, scores, targetDepth)
)

scores = [3, 5, 2, 9, 0, 7]
treeDepth = math.ceil(math.log2(len(scores)))
scores += [0] * (2 ** treeDepth - len(scores))   # pad to a complete tree

print("The optimal value is:", minimax(0, 0, True, scores, treeDepth))
print("Leaves evaluated:", leaf_count)
```

Output: The optimal value is: 5 | Leaves evaluated: 8

For alpha-beta, the counter goes in exactly the same three places: (1) declare the counter before the function, (2) increment it inside the base case `if depth == target:`, and (3) print it after the final call. The rest of the manual's alpha-beta code stays the same.

Results (verified by running both algorithms):

Tree	Minimax value	Minimax leaves	Alpha-Beta value	Alpha-Beta leaves
Tree A [3,5,2,9,0,7,0,0]	5	8	5	8
Tree B [3,5,2,9,3,5,2,9]	5	8	5	6

On Tree B, alpha-beta evaluates the leaves in the order 3, 5, 2, 9, 3, 5 and then stops. In the right MIN subtree, the first MAX child returns 5, which makes beta = 5. Since alpha from the root is already 5, the condition alpha >= beta is met and the whole second pair (2, 9) is never looked at. Two leaves are saved and the answer is still 5.

Task 3 — Reordering the Branches

The leaves of Tree A were rearranged in sorted order: [0, 0, 0, 2, 3, 5, 7, 9]. Minimax still returns 5 and still evaluates all 8 leaves, because plain Minimax always visits every leaf. Alpha-beta, however, now evaluates only 6 leaves instead of 8.

Ordering	Minimax value	Minimax leaves	Alpha-Beta leaves
Original [3,5,2,9,0,7,0,0]	5	8	8
Sorted [0,0,0,2,3,5,7,9]	5	8	6

Interpretation: reordering never changes the Minimax value — the game result is the same. It only changes how much work alpha-beta has to do. When the strong branches are examined early, alpha and beta tighten quickly and more branches are cut. With a bad ordering, alpha-beta can end up evaluating every leaf, exactly like plain Minimax.

Explanation (connection to the learning outcome)

This lab shows the core idea of adversarial search: an agent cannot just pick the move that looks best for itself, it must assume the opponent replies in the worst possible way. Minimax formalises this by alternating MAX and MIN levels. Alpha-beta keeps the exact same decision but avoids wasting time on branches that can no longer influence the root value, which is what makes game-playing programs fast enough to search deeper trees in real games.

Edge case / limitation found

On Tree A, alpha-beta evaluated all 8 leaves — the same as Minimax. The pruning gave no benefit at all for that ordering, because in the right subtree the value 7 appeared before the 0, so the bound never became tight enough to cut anything. This shows pruning is not guaranteed: it depends on move ordering. In the worst case alpha-beta does the same work as Minimax, and it never does more.

Reflection

What did I learn today? I learned how Minimax values are built bottom-up from the leaves, and that alpha and beta are simply the best scores found so far for MAX and MIN along the current path. Once alpha becomes greater than or equal to beta, the remaining branches cannot change the result.

How will I apply this in a real AI system? The same idea is used in game engines for chess, checkers and tic-tac-toe style problems, where the program must choose a move against an opponent. With pruning, the engine can search several moves deeper in the same time.

What failed or surprised me? I expected alpha-beta to always evaluate fewer leaves, but on the first tree it evaluated all 8, exactly like Minimax. That was surprising — the saving depends completely on the order in which branches are visited.

What would I do differently next time? I would test more orderings (best-first and worst-first) and record the leaf counts for each, so the effect of ordering is measured instead of just observed once.

Viva Preparation (trigger questions)

Q1. Why does MAX assume MIN will choose the worst outcome for MAX?
Because the opponent also plays to win. Planning against the opponent's best reply is the only safe strategy — any other assumption can be exploited.

Q2. What do alpha and beta represent?
Alpha is the best (highest) value MAX has found so far on the path from the root; beta is the best (lowest) value MIN has found so far. They are the bounds of the window that still matters.

Q3. Does pruning change the optimal Minimax result?
No. Pruning only removes branches that provably cannot affect the root value. The root value and the chosen move are exactly the same as plain Minimax.

Q4. Why does move ordering affect pruning efficiency?
If good moves are examined first, alpha and beta become tight early, so more later branches fail the alpha >= beta test and are cut. With poor ordering the bounds stay loose and little or nothing is pruned.